Table of Contents

Quick Start ---------- 2

Quick Start

The example code in this quickstart guide is for educational and demonstration purposes only. It may not represent best practices for production use.

Most functionality in this package comes from two components: `DepthPreprocessor` and `DepthMesher`. Below is a quick guide to getting them set up in your scene for runtime environment meshing on Meta Quest.

For full API details, see the reference documentation.

DepthPreprocessor

This component fetches depth frames from the Meta OpenXR API and exposes the depth texture to shaders. It is similar to Meta's `EnvironmentDepthManager`, but also exposes intrinsic frame data such as view and projection matrices. In addition, it generates and updates two RenderTextures:

- **Worldspace Positions**: 3D positions for each pixel in the depth texture, in world coordinates.
- **Worldspace Normals**: Surface normals for the corresponding points.

These textures feed into `DepthMesher` for real-time mesh generation.

This script **directly conflicts** with `EnvironmentDepthManager`, `AROcclusionManager`, and any other script using `XROcclusionSubsystem.TryGetFrame()`. It maintains partial compatibility by setting global shader variables such as `_EnvironmentDepthTexture` for Meta's occlusion shaders.

See the API reference for more details.

DepthMesher

`DepthMesher` consumes data from `DepthPreprocessor` to build a dynamic mesh using the Surface Nets algorithm. It can:

- assign the generated mesh to a `MeshFilter` for rendering,
- bake collision into a `MeshCollider` using a Jobs-based bake path,
- and bake a NavMesh with `NavMeshSurface`.

`DepthMesher` requires an instance of `DepthPreprocessor` in the same scene. It also requires a compute shader asset and an `OVRCameraRig`.

Runtime Data and Events

`DepthMesher` exposes the generated data and several lifecycle events:

- `Volume`: the TSDF volume used for meshing.
- `Mesh`: the generated mesh instance.
- `OnMeshDataUpdated`: invoked immediately after mesh data is updated.
- `OnBeforeColliderBuild`: invoked before optional collider baking starts.
- `OnAfterColliderBuild`: invoked after optional collider baking completes successfully.
- `OnBeforeNavMeshBuild`: invoked before optional NavMesh baking starts.
- `OnAfterNavMeshBuild`: invoked after optional NavMesh baking completes successfully.
- `OnMeshRefreshed`: invoked after mesh data is updated and all optional collision and NavMesh baking has completed.

Main Editor Variables

These key settings are exposed in the Inspector for tuning mesh quality, performance, and output behavior:

Variable	Description	Default	Constraints
Compute Shader	The compute shader containing kernels for volume updates and Surface Nets meshing. Required for the mesher to run.	None	Must be assigned
Volume Size	The dimensions of the TSDF volume grid (width × height × depth). Higher resolutions improve detail but increase memory usage and compute cost.	256 × 64 × 256	`Vector3Int`
Meters Per Voxel	The real-world size represented by each voxel, in meters. Smaller values yield finer detail but require larger volumes.	0.1 m	Min: 0
Min View Distance	The minimum distance from the camera at which depth data is considered for meshing. Closer user-occluded data is ignored.	1 m	Min: 0
Max View Distance	The maximum distance from the camera at which depth data is considered for meshing.	4 m	Min: 0

Variable	Description	Default	Constraints
Triangles Budget	The maximum number of triangles allowed in the generated mesh. This caps GPU memory usage and buffer sizes.	262144	Int
Target Volume Update Rate Hertz	The target update frequency for TSDF volume updates. Higher values improve responsiveness but increase GPU load.	45	Min: 0
Target Mesh Refresh Rate Hertz	The target refresh frequency for mesh generation. Lower values reduce CPU overhead in stable scenes.	1	Min: 0
OVRCameraRig	The camera rig providing the eye poses and tracking space. If not assigned, the component attempts to find one automatically.	Auto-found if possible	Must exist in scene
Mesh Filter Consumer	The `MeshFilter` that receives the generated mesh for rendering.	None	Optional
Mesh Collider Consumer	The `MeshCollider` that receives the generated mesh for physics collisions.	None	Optional
Bake Collision	Enables collider baking for optimized physics queries.	`true`	Boolean
Bake NavMesh	Enables NavMesh baking from the generated mesh.	`true`	Boolean
NavMesh Bake Subfrequency	Bake the NavMesh once every N mesh updates. A value of 1 bakes after every mesh update.	1	Min: 1
NavMesh Surface	The `NavMeshSurface` used for NavMesh baking. Required when NavMesh baking is enabled.	None	Required if baking is enabled
Use Fast NavMesh Bake	Uses an optimized NavMesh baking path that updates the NavMesh using	`false`	Boolean

Variable	Description	Default	Constraints
	only the generated depth mesh. This can be faster than a full `NavMeshSurface` bake, but all other NavMesh build sources are ignored.		
Fast NavMesh Bake Workers	Maximum number of worker jobs Unity may use during fast NavMesh baking. Higher values can reduce bake time on CPUs with more cores, but may increase CPU usage and compete with other jobs.	1	Min: 0

Behavior Notes

- The TSDF volume is automatically cleared on recenter via `OVRManager.display.RecenteredPose`. This ensures mesh accuracy because recentering resets the tracking space offset and invalidates prior voxel data.
- `DepthMesher` runs two asynchronous loops while enabled: one for TSDF volume updates and one for mesh refreshes.
- Mesh collision baking is performed asynchronously using Unity Jobs.
- NavMesh baking can use either the standard `NavMeshSurface.UpdateNavMesh` path or a faster build-source-driven path.

Fast NavMesh Bake Notes

When `Use Fast NavMesh Bake` is enabled:

- If the `NavMeshSurface` uses physics colliders, a `MeshCollider` consumer must be assigned.
- If the `NavMeshSurface` uses render meshes, a `MeshFilter` consumer, with a sibling `MeshRenderer`, must be assigned.
- The fast path only considers the generated depth mesh as a NavMesh build source.
- `Fast NavMesh Bake Workers` controls the maximum number of worker threads used for the bake.

Utility Component: CPUDepthSampler

See the API reference for details on using `CPUDepthSampler` to asynchronously sample world-space positions from depth data, for example for raycasting or occlusion checks.

Breaking Changes Notice

If you've just updated the package, check the changelogs for information on breaking changes.